Chapter 9. The test suite from scratch: every test, what it proves, and the code lines it exercises

This chapter is for a clean clone of the upstream project. Nothing from `labs/` is needed. You clone, install, build, run, and read this chapter next to the output.

Verification status. Every command below was run on 11 October 2026 on a fresh clone of `github.com/joshuaisaact/petri-flow` at commit `dcd6196` (Windows 11, Git Bash, Bun 1.3.14, turbo 2.8.3). Every line number was read from that clone. Expected outputs are copied from the real run. Where something was not run, the text says so.

Contents

1. From scratch: the commands and what you will see
2. How a test actually runs here (runner, workspaces, dist, SQLite, network)
3. The map: which test file proves which layer
4. Layer 0: petri-ts, the 208 lines that hold the mathematics
5. Engine package: 12 files, 125 tests
6. Workflows: 6 files, 40 tests, including the agent-benchmark properties
7. CLI: 4 tests, and what is not tested
8. Gate package: 3 files, 107 tests
9. Rules compiler: 67 tests
10. Vercel AI adapter: 64 tests
11. The other adapters: agent-sdk, claude-code, openclaw, pi-extension, pi-assistant
12. Examples (46) and comparisons (49)
13. The Windows failures, line by line, and the reversible patch
14. What the suite does not test
15. Reading order and how to present a result

Contents

Contents 2

1. From scratch: the commands and what you will see 4

1.1 What bun test prints on Windows, before any fix 4

1.2 One package at a time 4

1.3 bun test versus bun run test 5

2. How a test actually runs here 5

3. The map: which test file proves which layer 6

4. Layer 0: petri-ts, the 208 lines that hold the mathematics 6

5. Engine package: 12 files, 125 tests 7

5.1 types.test.ts (9 tests) 8

5.2 guard.test.ts (6 tests) 8

5.3 engine.test.ts (8 tests) 8

5.4 executor.test.ts (8 tests) 8

5.5 batch.test.ts (9 tests) 9

5.6 scheduler.test.ts (7 tests) 9

5.7 timeout.test.ts (6 tests) 9

5.8 expand.test.ts (14 tests) 10

5.9 analyse.test.ts (5 tests) 10

5.10 nodes.test.ts (36 tests) 10

5.11 sqlite-adapter.test.ts (5) and definition-store.test.ts (12) 10

6. Workflows: 6 files, 40 tests 10

6.1 coffee (5 tests) 11

6.2 order-checkout (5 tests) 11

6.3 github-lookup (6 tests) 11

6.4 contract-approval (8 tests) 11

6.5 simple-agent (7 tests) 11

6.6 agent-benchmark (9 tests): the properties, and what they do and do not prove 11

7. CLI: 4 tests, and what is not tested 12

8. Gate package: 3 files, 107 tests 13

8.1 gate.test.ts (48 tests) 13

8.2 compose.test.ts (46 tests) 14

8.3 replay.test.ts (13 tests) 14

9. Rules compiler: 67 tests 15

10. Vercel AI adapter: 64 tests 16

11. The other adapters . . . 17

11.1 packages/agent-sdk (21 tests, src/gate.test.ts) . . . 17

11.2 packages/claude-code (18 tests, src/__tests__/hook.test.ts) . . . 17

11.3 packages/openclaw (20 tests, src/__tests__/index.test.ts) . . . 17

11.4 packages/pi-extension (19 tests, src/__tests__/compose.test.ts) . . . 18

11.5 packages/pi-assistant (59 tests, 4 files) . . . 18

12. Examples (46 tests) and comparisons (49 tests) . . . 18

12.1 examples/0X/agent.test.ts . . . 18

12.2 comparisons/n8n/comparison.test.ts (11 tests) . . . 19

12.3 comparisons/openclaw-safety/src/scenarios.test.ts (38 tests) . . . 19

13. The Windows failures, line by line, and the reversible patch . . . 19

13.1 The patch . . . 20

14. What the suite does not test . . . 20

15. Reading order and how to present a result . . . 21

1. From scratch: the commands and what you will see

All commands are Git Bash, run from the clone root.

```
git clone https://github.com/joshuaisaact/petri-flow
cd petri-flow
bun install          # observed: "435 packages installed [41.46s]"
bun run build        # observed: ~31 s; turbo builds every package into dist/
bun test             # observed: ~20 s
```

Two things you will notice and should not worry about:

- `bun install` rewrites `bun.lock` (the repo pins `"packageManager": "bun@1.2.7"` in `package.json`, your Bun is newer). `git status` shows `M bun.lock`. Do not commit it.
- `bun run build` is required before `bun test`. Section 2 explains why. Without it the first failure is `Cannot find module "@petriflow/gate"`.

1.1 What `bun test` prints on Windows, before any fix

```
 588 pass
 9 fail
 4 errors
 2407 expect() calls
Ran 597 tests across 39 files. [19.94s]
```

Those 13 problems are all Windows path handling, in two places (section 13). After the reversible six-file patch in section 13 the same command prints:

```
 639 pass
 0 fail
Ran 639 tests across 39 files. [19.56s]
```

The suite defines 639 tests in 39 files. The unpatched Windows run reports 597 because the four `examples/*/agent.test.ts` files crash while loading (their 46 tests are never registered; each file counts as one failure plus one error).

1.2 One package at a time

Run a directory to see its own numbers. Observed on the fresh clone, unpatched, Windows:

Command	Tests	Observed
`bun test packages/engine`	125 in 12 files	125 pass
`bun test packages/gate`	107 in 3 files	104 pass, 3 fail (path)
`bun test packages/rules`	67 in 1 file	67 pass
`bun test packages/vercel-ai`	64 in 2 files	64 pass
`bun test packages/cli`	4 in 1 file	4 pass
`bun test packages/agent-sdk`	21 in 1 file	21 pass
`bun test packages/claude-code`	18 in 1 file	18 pass, only if `C:\tmp` exists (section 13)

Command	Tests	Observed
`bun test packages/openclaw`	20 in 1 file	20 pass
`bun test packages/pi-extension`	19 in 1 file	19 pass
`bun test packages/pi-assistant`	59 in 4 files	57 pass, 2 fail (path)
`bun test workflows`	40 in 6 files	40 pass (18 s: real timers, one live network call)
`bun test examples`	46 in 4 files	0 pass, 4 fail, 4 errors (file URL path)
`bun test comparisons`	49 in 2 files	49 pass

Total 639. Other useful forms:

```
bun test packages/rules/src/__tests__/compiler.test.ts     # one file
bun test packages/rules -t "limit A to N"                  # tests whose name matches
bun test workflows/agent-benchmark                          # the 9 property tests, prints a
    summary
```

1.3 `bun test` versus `bun run test`

They are different commands.

- `bun test` is Bun's own runner. It finds every `*.test.ts` under the current directory and runs them in one process. The numbers above come from it.
- `bun run test` runs the `test` script in the root `package.json`, which is `turbo run test`. Turbo runs each workspace package's own `bun test` as a separate task, builds dependencies first (`turbo.json`: `"test": { "dependsOn": ["^build"] }`), caches results, and by default **stops scheduling after the first failing package**. On the fresh Windows clone it printed `Tasks: 8 successful, 21 total` and `Failed: @petriflow/gate#test`. With `bun run test -- --continue` it ran everything: `Tasks: 19 successful, 22 total`, `Failed: @petriflow/examples#test, @petriflow/gate#test, @petriflow/pi-assistant#test`.

For reading results, `bun test` is simpler. For CI, turbo's dependency ordering is the point.

2. How a test actually runs here

Runner. Every test file imports `describe`, `it` (or `test`) and `expect` from `bun:test`. There is no Jest or Vitest configuration anywhere; Bun compiles TypeScript on the fly and runs the file. A test is an `it("name", fn)`; it passes if `fn` returns (or resolves) without a failed `expect`.

Workspaces. The root `package.json` declares workspaces `packages/*`, `workflows/*`, `comparisons/*`, `apps/*`, `examples`. `bun install` links each workspace into `node_modules/@petriflow/<name>` as a symlink to its folder. So `import ... from "@petriflow/engine"` inside a test resolves to `packages/engine`.

Why build first. Each package's `package.json` has an `exports` map pointing at `./dist/index.js`. A test in `packages/gate` imports its own code from `./gate.js` (source), but the moment anything imports `@petriflow/engine` or `@petriflow/gate` by package name, the compiled `dist/` file is loaded. That is why `turbo.json` makes `test` depend on `^build`, and why a fresh clone needs `bun run build` once. The sources you read in this chapter are `src/*.ts`; what executes for cross-package calls is the equivalent code in `dist/`.

Storage. Engine and workflow tests use `new Database(":memory:")` from `bun:sqlite`, so nothing is written to disk by them. Two test files touch the filesystem: the rules `loadRules` test writes one file into the OS temp directory and deletes it, and the Claude Code hook test writes state files under `/tmp` (section 13).

Network and models. No test calls a language model. One test calls the real network: `workflows/github-lookup/github-lookup.test.ts` has a `describe("live")` block (line 116) that fetches `api.github.com/users/octocat`. If the laptop is offline or the proxy blocks it, that single test fails and nothing else changes. The example agents (`examples/*/agent.ts`) do call Anthropic models, but the test files never import them; they import only the `.rules` file next to them.

Timers. Workflow tests use real `setTimeout` delays inside `execute` functions (coffee waits 2 s, 1 s, 3 s; order-checkout 0.8 s and 2.5 s). That is why `bun test workflows` takes about 18 s and why `coffee.test.ts` starts with `setDefaultTimeout(15_000)`.

3. The map: which test file proves which layer

```
petri-ts (node_modules, 208 lines)      <- engine/src/types.test.ts, cli/test/analyse.test.ts
   |
packages/engine  (defineWorkflow, guards, executor, scheduler, SQLite, analyse, nodes)
   |   <- engine/src/*.test.ts (12 files), workflows/*/*.test.ts (6), cli/test (1)
   |
packages/gate    (SkillNet, handleToolCall, composedToolCall, manager, replay, autoAdvance)
   |   <- gate/src/{gate,compose,replay}.test.ts
   |
packages/rules   (text rules -> SkillNets + state counts)     <- rules compiler.test.ts
   |
adapters: vercel-ai, agent-sdk, claude-code, openclaw, pi-extension, pi-assistant
   |   <- one or more test files each
   |
examples/*.rules + comparisons/*                  <- examples/*/agent.test.ts, comparisons/*
```

Two independent roads meet at `packages/engine`: the **workflow road** (engine, workflows, CLI) and the **gate road** (gate, rules, adapters, examples). The gate road imports only `canFire` and `fire` from the engine, which re-exports them from petri-ts. Nothing on the gate road calls `reachableStates` at runtime; only the rules compiler calls `analyse` once, to count states (section 9).

4. Layer 0: petri-ts, the 208 lines that hold the mathematics

File: `node_modules/petri-ts/dist/index.js` (version 0.2.0 at the root; the gate package additionally carries a nested `@petriflow/engine` 0.1.3 with petri-ts 0.1.6, see chapter 2). Every Petri-net operation in the repo is one of these functions:

Lines	Function	What it does
2-11	`canFire(marking, t)`	counts how many tokens each input place needs (a place listed twice needs two) and checks the marking has them
12-20	`fire(marking, t)`	throws if not enabled, else copies the marking, subtracts one token per input, adds one per output
62-97	`reachableStates(net, limit = 10000)`	breadth-first search from the initial marking; a state key is `JSON.stringify` over sorted place names (line 73-74); throws "Reachable state space exceeded 10000 states" (85-88)

Lines	Function	What it does
98-100	`enabledTransitions`	filter by `canFire`
101-105	`terminalStates`	reachable markings with no enabled transition
106-108	`isDeadlockFree`	`terminalStates.length === 0`
109-122	`checkInvariant(net, weights)`	the weighted token sum is the same in every reachable marking
123-138	`analyse(net, options)`	runs the above once and returns counts, terminal states, invariant results, optional DOT text

Tests that prove exactly this layer:

- `packages/engine/src/types.test.ts`, describe **"structural compatibility"** (5 tests). The test builds a `WorkflowNet` with guards and passes it straight into `reachableStates`, `terminalStates`, `isDeadlockFree`, `enabledTransitions`, `checkInvariant`, `toDot`, `canFire`, `fire`. It passes because `WorkflowTransition` is declared as an intersection type `Transition<Place> & {...}` (`packages/engine/src/types.ts:20-28`), so the extra fields are ignored by petri-ts. Note the assertion `isDeadlockFree(net)` is `false` for a net that terminates: in petri-ts "deadlock free" means "never stops", which is why the engine adds its own notion of valid terminal states (section 5, `analyse.ts`).
- `packages/cli/test/analyse.test.ts` (4 tests): a three-place pipeline has exactly 3 reachable states; an invariant with weights 1,1,1 holds and one with weights 2,1,1 does not; `dot: true` yields text containing `digraph`; a two-place cycle is deadlock free with zero terminal states.

What this layer does not contain: symbolic analysis, coverability, liveness, place invariants computed from the structure (weights are supplied by the caller), and any notion of guards. `toNet` in the engine strips guards before analysis (`packages/engine/src/workflow.ts:279-290`), so every analysis in this repo is over the structure only.

5. Engine package: 12 files, 125 tests

Sources: `packages/engine/src/`. The reading order that makes the tests obvious:

File	Lines	Role
`types.ts`	64	`WorkflowTransition` (20-28), `WorkflowDefinition` with `guards` and `executors` maps (44-55)
`guard.ts`	8	`compileGuard(expr)`: compiles a filtrex expression; the guard sees `ctx` fields plus `marking` (6-7)
`workflow.ts`	290	`defineWorkflow` validation (43-81), guard compilation (84-89), executor extraction (92-100), node registry (103-113); `expandWorkflow` (134-259); `injectTokens` (265-273); `toNet` (279-290)
`engine.ts`	85	`canFireWorkflow` = `canFire` AND guard (24-27); `enabledWorkflowTransitions` (34-51); `fireWorkflow` fires, then runs `execute`, then shallow-merges the returned patch into context (71-77)
`executor.ts`	232	`createExecutor`: `step()` picks one enabled transition, asks the decision provider only when more than one is enabled (122-137) and falls back to the first if the provider names an unknown one (137); `stepBatch()` greedily reserves tokens (187-197), runs executors in parallel (200-206), merges patches in name order (209-213)

File	Lines	Role
`scheduler.ts`	345	`createInstance` (56-67), `tick()` phases: expired timeouts (85-100), step (190-194), timeout bookkeeping (196-223), idle to completed (226-236), persist and record history (254-275), error to `failed` (281-296); `injectToken` (305-314); `updateExecutor` (316-318)
`analyse.ts`	63	wraps petri-ts `analyse`, merges `definition.invariants` (33-36), splits terminal states into valid and unexpected using `terminalPlaces` (44-55)
`nodes.ts`	103	`isPrivateHost` (14-43), `httpNode` with the private-host block (55-57) and a 30 s `fetch` (62-68), `timerNode` (80-96)
`persistence/schema.ts`	41	three tables: `workflow_instances`, `transition_history`, `timeout_entries`
`persistence/sqlite-adapter.ts`	278	creates the tables at construction (49-51); `listActive` selects `status = 'active'` for this workflow name (56-58)

5.1 `types.test.ts` (9 tests)

- **"structural compatibility"** (5): section 4.
- **"defineWorkflow"** (4): the validator at `workflow.ts:43-81` throws `unknown output place "unknown"`, `unknown place "ghost"` (initial marking), and `Terminal place "donee" is not a known place`. These are the only three places a malformed definition is rejected; a transition with a typo in its name is not an error.

5.2 `guard.test.ts` (6 tests)

`compileGuard` from `guard.ts:3-8`. Tests: `waterTemp >= 90`, a bare boolean field, `paid and approved`, `marking.inventory == 0` (works because line 7 spreads `ctx` and adds `marking`), the constant `0` is false, and `@@@` throws at compile time. The guard language is filtrex, not JavaScript; it cannot call functions or mutate anything.

5.3 `engine.test.ts` (8 tests)

Definition at lines 14-44: `submit`, `approve` (guard `amount < 10000`, has `execute`), `reject` (guard `amount >= 10000`).

- **"canFireWorkflow"** (4): `engine.ts:24` rejects when the marking lacks tokens; line 26 rejects when the guard says no; no guard means structural only.
- **"enabledWorkflowTransitions"** (1): with one token in `review`, amount 5000 enables only `approve`, amount 15000 only `reject`. This is the mechanism that makes guards act as branches.
- **"fireWorkflow"** (3): line 71 fires, lines 74-77 run `execute` and merge `{ approved: true }`; without `execute` the context object is returned unchanged; an unenabled transition throws `Cannot fire workflow transition: approve` (line 68).

5.4 `executor.test.ts` (8 tests)

- `step()` returns `{ kind: "fired", transition, marking, context, terminal }` or `{ kind: "idle" }` (`executor.ts:115-117, 153-160`). `terminal` is true when nothing is enabled after firing (line 158).
- Errors thrown by `execute` propagate out of `step()` (test "propagates execute errors as throws"); the scheduler is what turns that into `status: "failed"`.

- **Decision provider**: with two enabled transitions and a provider that answers `go_b`, `step()` fires `go_b` and reports `decision: { reasoning, candidates: ["go_a","go_b"] }` (lines 125-141). If the provider answers a name that is not enabled, line 137 falls back to the first enabled transition. This is the only place a model can influence the engine (lab 4 in chapter 7 uses exactly this hook), and the fallback is why a model cannot invent a transition.

5.5 `batch.test.ts` (9 tests)

`stepBatch` (`executor.ts:163-230`) and `Scheduler.tick({ maxConcurrent })`.

- Two independent branches fire together; two transitions competing for one token do not both fire ("first-enabled wins", lines 190-197, the working marking is advanced with `fire` so the second cannot reserve the same token).
- Context patches merge in alphabetical transition-name order (line 209), so `z_second` overwrites `a_first` for the same key. The test "context patches merge in transition-name order" asserts `last === "z"`.
- `maxConcurrent: 1` is identical to `step()`; `0` or `-1` throws (line 169-171).
- Scheduler: `tick({ maxConcurrent: 10 })` fires 2 and completes in one tick; plain `tick()` fires one per tick (scheduler.ts:103 chooses the path).

5.6 `scheduler.test.ts` (7 tests)

Pipeline `begin`, `process`, `complete` with an in-memory SQLite adapter.

- "drives a workflow to completion via tick()": five ticks fire exactly three transitions, `onFire` order is `begin, process, complete`, final `status` is `completed`, context log is the three strings. Each tick does: `listActive()` (scheduler.ts:78), `loadExtended` (81), `executor.step` (190), `saveExtended` with the new status (254-259), `recordTransition` (261-269), `onFire` (271).
- "handles multiple instances": two instances in one DB, both completed.
- "guards block transitions correctly": score 50 fires `deny` only.
- "handles execution errors gracefully": `execute` throws `boom`; `onError` receives it and the instance is saved with `status: "failed"` (lines 281-296). Note the marking is the pre-fire marking; nothing is rolled back in the executor because nothing was committed.
- "injectToken adds tokens and reactivates completed instances": a transition needing two tokens is idle, the instance is marked completed (226-234, because no timeout is pending), `injectToken` sets `status: "active"` again (305-314), the next tick fires it. This is the engine's way to receive an external approval.
- "rejects duplicate instance IDs": `createInstance` line 57-59.
- "records transition history": three rows with before/after markings (`transition_history` table, schema.ts:19-30).

5.7 `timeout.test.ts` (6 tests)

A `manual` transition `approve` guarded by `approved` with `timeout: { place: "timed_out", ms }`; a second transition `handle_timeout` needs `waiting` and `timed_out`.

- "fires timeout when guard blocks transition": tick 1 is idle but `getTimeoutCandidates` (executor.ts:93-101, structural enablement only) lists `approve`, so a timeout row is scheduled (scheduler.ts:214-223) and the instance stays `active` (247-253, `hasPendingTimeouts`). After 150 ms, tick 2 finds the expired row (85-97), injects a token into `timed_out`, fires `onTimeout`, and `handle_timeout` fires in the same tick.

- "normal fire cancels timeout": the context is edited directly through the adapter to set `approved: true`; `approve` fires; `clearTimeouts` (line 199) removes the row; a later tick fires no timeout.
- "competing transition consumes tokens and clears timeout": when a different transition consumes the token, the candidate set shrinks and lines 206-211 clear the stale entry.
- Plus: pending timeouts keep an instance active; candidates depend on the marking; a timeout naming an unknown place is rejected at `defineWorkflow` (workflow.ts:58-62).

These tests use real `sleep` of 150 to 250 ms; they are the slow part of the engine suite.

5.8 `expand.test.ts` (14 tests)

`expandWorkflow` (workflow.ts:134-259) returns a new definition with extra transitions and places, re-validates everything, rejects duplicate names (168-175), and does not mutate the original (test "does not mutate the original definition" counts transitions, guards, executors and terminal places before and after). `injectTokens` is the pure version of `Scheduler.injectToken`. **"Scheduler.updateExecutor"** (2 tests): swapping the executor inside an `onFire` callback does not affect the running tick because `tick()` snapshots `this.executor` at line 75; the next tick sees the expanded net. This is the closest thing in the repo to runtime self-modification of a workflow, and it is deliberately tick-atomic.

5.9 `analyse.test.ts` (5 tests)

`analyse.ts:44-55`: with `terminalPlaces: ["b"]`, a terminal marking `{a:0,b:0,c:1}` is classified as unexpected (the "stuck" case the CLI's `--strict` reports); with empty `terminalPlaces` every terminal state is valid (line 51-54). Invariants declared on the definition are checked automatically (33-36). This file is where "deadlock" in this repo is defined: a terminal marking with no token in any declared terminal place.

5.10 `nodes.test.ts` (36 tests)

- **"httpNode"** (5) and **"timerNode"** (4): config validation and execution; the http tests replace `globalThis.fetch` with a stub and check the namespaced result `{ [transitionName]: { status, ok, data } }` (nodes.ts:74-76).
- **"defineWorkflow with nodes"** (7): a transition with `type: "http"` and a `config` but no `execute` gets an executor compiled from the node registry (workflow.ts:103-113); an explicit `execute` wins; unknown types (`automatic`, `manual`) are skipped; bad config throws during `defineWorkflow` because `node.validate` runs at line 109.
- **"isPrivateHost"** (14) and **"httpNode SSRF"** (4): loopback, RFC1918, link-local, CGNAT, IPv6 forms, and the cloud metadata address are refused before any request (nodes.ts:55-57). This is the only security control in the engine package.

5.11 `sqlite-adapter.test.ts` (5) and `definition-store.test.ts` (12)

The adapter round-trips with petri-ts's own `createDispatcher`, saves and reloads marking, context and status, lists only `active` instances, and throws `Instance not found`. The definition store serializes a definition to JSON (places, transitions with `guard` strings, `timeout`, `config`, invariants), stores it in a fourth table, and `defineWorkflow(loaded)` recompiles the guards from their strings. `execute` functions are not serializable and are dropped (test "extracts serializable fields": the `go` transition is stored without `execute`).

6. Workflows: 6 files, 40 tests

Each workflow folder has `definition.ts`, an `index.ts` that re-exports it and runs a demo when executed directly (`if (import.meta.main)`), and one test file. Every test file follows the same two-part pattern: run

the thing with the Scheduler and check what fired; then analyse the net and check structural properties.

6.1 `coffee` (5 tests)

Three transitions: `heatWater` (2 s), `grindBeans` (1 s), `pourOver` (needs three tokens, guard `waterTemp >= 90`, 3 s). "brews coffee successfully" ticks until idle and asserts the context (`waterTemp 96`, `grindSize medium`, `brewed true`) and `coffeeReady: 1`. "all paths produce coffee" uses `terminalStates(toNet(...))`: exactly one terminal marking, and it has `coffeeReady`. "heatWater and grindBeans are concurrent" only asserts more than 2 reachable states (the CLI shows 5). This is the net the website uses to introduce concurrency.

6.2 `order-checkout` (5 tests)

`reserve_stock` consumes `order_placed` and one `inventory` token; `process_payment` consumes `payment` and `reserved`; `out_of_stock` has guard `marking.inventory == 0`. "inventory tokens are conserved" calls `checkInvariant` with weights `inventory 1, reserved 1, shipped 1`; the same weights are declared on the definition, so "passes analyse" reads `result.invariants[0].holds === true`. This is the invariant you see in `petriflow analyse workflows/order-checkout/index.ts` (chapter 7, demo 1). "rejects order when no inventory" rebuilds the definition with `inventory: 0` and checks only `out_of_stock` fires.

6.3 `github-lookup` (6 tests)

Two `http` transitions compiled from `config` (no `execute`). "both transitions have http node executors compiled" and "no explicit execute functions" prove the node registry path. "runs to completion with mocked fetch" stubs `globalThis.fetch` and checks the two namespaced results in context. **"fetches real data from GitHub API" is a live network call** (`describe("live")`, line 116). Offline, this is the one test that fails for a reason that is not Windows.

6.4 `contract-approval` (8 tests)

`submit` forks into `awaitingFinance` and `awaitingLegal`; each can approve or reject; `execute` needs both approvals. The tests enumerate terminal states and assert: any terminal state with `financeRejected > 0` has `executed === 0` (same for legal), there are exactly 4 terminal states, no unexpected terminal states, and reachability contains both interleavings (finance first and legal first), which is the definition of concurrency in a Petri net. The declared invariant on the finance track holds.

6.5 `simple-agent` (7 tests)

`plan`, `dispatchTool`, `completeTool`, `generate`, `iterate` (consumes `iterationBudget`, initial 2). "drives agent to completion via tick()" runs the engine with no decision provider: at `resultsReady` both `generate` and `iterate` are enabled and `step()` takes the first in definition order (`generate`), so the run completes after one pass. Lab 4 in chapter 7 plugs a model into exactly this choice. Property tests: every terminal state has `responseGenerated`, the budget stays within `[0, 2]` in every reachable state, and when the budget is 0 the `resultsReady` state is not terminal (because `generate` is still enabled).

6.6 `agent-benchmark` (9 tests): the properties, and what they do and do not prove

The net (`workflows/agent-benchmark/definition.ts`): `plan` to `distribute`, which forks into three decision places; each branch either dispatches (guard `useSearch`, `useDB`, `useCode`) or skips; the code branch goes `dispatchCode` to `codePending` to `requestApproval` to `humanApproval`, then `approveCode` (manual) to `codeApproved` to `executeCode`, or `rejectCode` (manual) straight to `codeDone`; `joinResults`

needs `searchDone`, `dbDone`, `codeDone`; then `generate`, or `iterate` which consumes one `iterationBudget` token (initial 3) and returns to `userQuery`.

The test file computes `reachableStates`, `terminalStates` and `analyse` once at describe time and then asserts:

Test	Kind	What is checked
terminates: all terminal states are responseGenerated	reachability	every terminal marking has `responseGenerated === 1`
no unexpected terminal states (no deadlocks)	reachability	`unexpectedTerminalStates` is empty
code execution always passes through humanApproval	**structural**	for every transition whose inputs include `codePending`: outputs exclude `codeDone` and include `humanApproval` (test lines 28-34)
no reachable state has codeDone without humanApproval	reachability	no marking has `codePending > 0` and `codeDone > 0` at once
joinResults requires all three tool completion tokens	structural	the `joinResults` inputs are exactly the three done places
no terminal state has orphaned pending work	reachability	eight pending places are 0 in every terminal marking
reachable state space is finite	reachability	count between 1 and 9999
iteration budget limits loops to 3	reachability	`0 <= iterationBudget <= 3` in every marking
iterate cannot fire when budget is exhausted	reachability	`resultsReady > 0 && budget == 0` is never terminal

The `afterAll` prints what you will see at the end of `bun test workflows/agent-benchmark`:

```
=== Agent Benchmark Analysis ===
  Reachable states: 196
  Terminal states:  4 (all valid)
  Unexpected terminal states (deadlocks): 0
  Iteration budget: 3
  Terminal state variants by remaining budget:
    budget=3: 1 variant(s)
    budget=2: 1 variant(s)
    budget=1: 1 variant(s)
    budget=0: 1 variant(s)
```

Two things to say out loud in a review. First, "code execution always passes through humanApproval" is a check over the transition list, not over reachability; that is exactly why chapter 7's demo 3 (adding a `bypassApproval` transition) is caught by this test while `petriflow analyse --strict` still exits 0 with 196 states. Second, the "human gate" the tests prove is that the net cannot reach `codeDone` without visiting `humanApproval`; nothing in these tests says who or what fires the `manual` transition. In the Scheduler, a `manual` transition is fired like any other as soon as it is enabled (`executor.ts:122-123` does not look at `type`), and the test "drives agent to completion" in simple-agent relies on that. "Manual" is enforced by the gate package (section 8), not by the engine.

7. CLI: 4 tests, and what is not tested

`packages/cli/test/analyse.test.ts` imports `analyse` and `defineWorkflow` from the engine. It never runs `packages/cli/src/cli.ts`. Therefore the following lines have no test:

- `cli.ts:24-29`: the only command is `analyse` (and the spelling `analyze`); anything else prints `Unknown command` and exits 1. There is no `verify` command.
- `cli.ts:71-89`: `--strict` exits 1 only for unexpected terminal states or a declared invariant that does not hold. It does not call `isDeadlockFree`, and it checks no property that the definition did not declare.
- `format.ts:21-72`: the text layout you see in demo 1, including the `(all valid)` suffix.

Chapter 3 and chapter 5 (loophole on "verified") rest on these untested lines; read them once, they are short.

8. Gate package: 3 files, 107 tests

Sources in `packages/gate/src/`. The order to read them:

File	Lines	Role
`types.ts`	81	`GatedTransition` with `type: "auto" or "manual"`, `tools`, `deferred` (11-26); `SkillNet` with `freeTools`, `toolMapper`, `validateToolCall`, `onDeferredResult`, `ruleMetadata` (32-74); `defineSkillNet` is the identity function (77-81), it validates nothing at runtime
`events.ts`	23	`GateToolCall`, `GateToolResult { isError }`, `GateContext { hasUI, confirm }`, `GateDecision`
`advance.ts`	72	`autoAdvance`: fire every enabled structural transition (`auto` with no tools, line 6-8) until nothing changes; competing structural transitions are left alone (53-56); cycle detection by seen markings (66-68)
`gate.ts`	185	single-net path: `resolveTool` applies the mapper (9-20); `handleToolCall` (72-144): free tools pass (81-83), no enabled matching transition blocks (88-93), `validateToolCall` (98-106), manual approval (108-123), re-check after the `await` (125-131), deferred records `pending` (133-137), otherwise fire and auto-advance (140-141); `handleToolResult` (150-185): error leaves the marking alone (160-163), success fires the pending transition (166-167) and calls `onDeferredResult` (170-180)
`compose.ts`	187	`classifyNets` gives each net a verdict: free, abstain (no transition mentions the tool, 63-64), blocked (jurisdiction but nothing enabled, 71-77), gated (79); `composedToolCall` is the four-phase handler: any blocked rejects (102-104), none gated allows (112-114), manual approvals (117-137), validators with meta rollback (141-160), commit with a last `canFire` re-check for non-deferred transitions (164-171) and firing or pending (173-184)
`manager.ts`	258	`createGateManager` wraps the handler for `onDecision` and shadow mode (52-62); array manager (113-164) and registry manager with `addNet`/`removeNet` (166-249); `replayNets` (75-111); `formatPromptForNets` builds the text that goes into the model's system prompt (251-258)
`format.ts`	36	block reasons from `ruleMetadata`, fallback `Tool 'x' is not available in the current state.` (35)
`find-blocking-net.ts`	20	re-classifies to name the net that blocked

8.1 `gate.test.ts` (48 tests)

Uses a five-place `simpleNet` plus three hand-built nets imported from `packages/pi-extension/src/nets/`: `toolApprovalNet`, `implementNet`, `nukeNet`.

- **"autoAdvance"** (7): fires `setup` from the initial marking; stops in front of a tool-gated transition; cascades `post`; leaves an ambiguous choice (`goLeft`/`goRight`) untouched (advance.ts:53-56); terminates on a self-loop and on a two-place cycle (66-68); an AND-join waits for all inputs.
- **"handleToolCall"** (9): free tools pass even with an empty marking; a gated tool with no enabled transition blocks; a gated tool fires and the marking moves (and auto-advances through `post`, so `d` gets the token, test "allows gated tool when matching transition is enabled"); human rejection blocks with "rejected by human review"; approval fires; **the TOCTOU guard**: a `confirm` callback that mutates the marking while awaited makes line 126 block afterwards; no UI blocks with "requires human approval".
- **"getEnabledToolTransitions"** (2), **"formatMarking"** (1): `b:2, c:1` prints only non-zero places.
- **"toolApprovalNet"** (5): `bash`, `write`, `edit` are `manual` self-loops on `ready`, so each call needs approval and the token returns.
- **"implementNet"** (6): `toolMapper` turns `bash` with `git commit` into `git-commit` and `git push` into `git-push`; plain `bash` is free; push is blocked before commit and allowed after; a full cycle returns to `working`.
- **"path extraction"** (2), **"pathCovers"** (5), **"nukeNet"** (11): a bash command is classified as `backup` or `destructive` by regex tables (nuke.ts:10-38); a backup is deferred and records its path in `state.meta.backedUpPaths` on success (`onDeferredResult`, 150-158); a destructive command must be covered by a recorded backup (`validateToolCall`, 161-181), which then consumes that entry. Three of these tests fail on Windows before the patch (section 13).

8.2 `compose.test.ts` (46 tests)

Synthetic nets A to E plus two validator nets.

- **"classifyNets"** (5): free, gated, abstain, blocked, and independence across nets.
- **"createGateManager, array mode"** (8): free in all nets allows; one blocked net rejects all (compose.ts:102-104); abstain plus allow allows; deferred is allowed; manual approval approved, rejected, and no UI.
- **"deferred transitions"** (4): the marking does not change until the result; success fires; failure does not; a result fires only in the net that recorded the pending entry.
- **"meta rollback"** (2): validator A mutates `meta`, validator B blocks, so A's mutation is restored from the snapshot taken at line 141; a validator that mutates and then rejects itself is rolled back too (lines 154-156, `j <= i`).
- **"TOCTOU re-check"** (1): in composed mode the marking is re-checked at commit (164-171) with the reason `no longer available (state changed)`.
- **"single net equivalence"** (2), **"system prompt and status"** (3): the prompt header is `## Active Petri Nets (composed)` (manager.ts:257); each section lists the gated tools that are enabled right now and the marking.
- **"real nets composition"** (3): `communicateNet` plus `cleanupNet` from pi-assistant; `rm -rf build/` is blocked structurally (no `backedUp` token) before any path check.
- **"dynamic management"** (12): registry mode; `addNet`/`removeNet` messages; a removed net keeps its state; a pending deferred result still lands on a removed net because `handleToolResult` fans out to the whole registry (manager.ts:187-191).
- **"shadow mode"** (6): a block becomes an allow (manager.ts:57-59) but `onDecision` still sees `block: true`.

8.3 `replay.test.ts` (13 tests)

`manager.replay(entries)` (manager.ts:75-111): for each entry with `isError: false`, resolve the tool, skip free tools and tools no enabled transition matches, otherwise `fire` directly and auto-advance. Tests: a sequence advances; failed entries are skipped; `string[]` shorthand; idempotent when the transition cannot fire twice; multi-step; unknown and free tools skipped; `toolMapper` applied when `input` is present; multiple nets; a replayed state unlocks a later live call; `onDeferredResult` is invoked for deferred transitions; registry mode.

Read lines 94-96 once: replay fires transitions without `validateToolCall`, without manual approval, and without re-checking anything about where the entries came from. Chapter 5's forged-history loophole and lab 3's spoof script are this function being handed a client's message history.

9. Rules compiler: 67 tests

Source: `packages/rules/src/compiler.ts` (439 lines). The test file is `packages/rules/src/__tests__/compiler.test.ts`.

Lines	What	Shape produced
36-61	`stripComments`, `escapeRegex`, `parsePattern` (`/re/` raw, bare word becomes `\bword\b`)	
63-171	`parseLine`: `map`, `require human-approval before`, `require A before B`, `limit A to N per scope`, `block A`; every error message names the line number	
177-204	`compileSequence`	places `idle, ready, gate`; `do-A` is `deferred` (190), `ready` to `gate`; `do-B` `gate` to `ready`
206-225	`compileApproval`	`approve` is `manual`, `ready` to `ready`
227-246	`compileBlock`	`do-A` needs `locked`, which starts at 0 and is never produced
248-296	`compileLimit`	session: `do-A` consumes `ready` and `budget`; per action: adds `spent` and a `refill` transition that consumes `ready` and `spent`
306-352	`buildToolMapper`: `map` statements first, then dot notation `tool.action`	
372-396	`toPetriNet` and `verifyNets`: `analyse(...)` per net, keep only `reachableStateCount`	
408-433	`compile`: parse lines, compile rules, attach mappers, verify	
436-439	`loadRules(path)`: read a file, `compile`	

Describe blocks and what each proves:

- **"compile, parser"** (13): names of the produced nets (`require-backup-before-delete`, `approve-before-deploy`, `block-rm`, `limit-deploy-3`, `limit-push-1-per-test`); comments and blanks stripped; array input; six rejection messages (`unknown keyword 'allow'`, `expected 'before' at position 3, got 'after'`, and so on).
- **"require A before B"** (5): run through the single-net `handleToolCall`: A allowed and deferred; B blocked; B allowed after A's success result; after B fires the net is back at `ready` so B is blocked again;

A's failure does not unlock B. The reasons are produced from `ruleMetadata` by `format.ts`: `delete requires a successful call to backup first.`

- **"require human-approval before B"** (3), **"block A"** (2), **"limit A to N per session"** (1: three allowed, fourth blocked), **"limit A to N per action"** (1: push, push blocked, test refills, push again).
- **"multiple rules compose via GateManager"** (1): `rm` is blocked by `block-rm`, `delete` by the sequence net, `backup` is allowed because the block net abstains.
- **"compile, dot notation parser"** (6) and **"dot notation, action dispatch"** (6): a rule naming `discord.sendMessage` gets a mapper that joins `toolName` and `input.action`; `discord` with `action: "react"` passes because no net has jurisdiction over `discord.react`.
- **"compile, map statement parser"** (8) and **"map statements, bash command gating"** (7) and **"map with bare word patterns"** (6): `map bash.command /\brm\s/ as delete` turns a shell command into a virtual tool; bare words are word-bounded (`format disk` does not match `rm`); `map` wins over dot notation. Chapter 5, loophole 2 shows what these regexes do not catch.
- **"compile, verification"** (6): exactly what `verification` contains. `block rm` reports 2 states, `limit deploy to 3 per session` reports 5 states (`idle`, then budget 3, 2, 1, 0). Nothing else is asserted because nothing else is computed: `verifyNets` returns only `{ name, reachableStates }` (lines 383-396). The website's "formally verified" is these two numbers.
- **"loadRules"** (2): writes `test-safety.rules` into `os.tmpdir()` (works on Windows), reads it back, and rejects a missing file.

Note what the per-action limit test does not cover: it calls `push` first, then `test` to refill. Chapter 5, loophole 5 (the website's WhatsApp example) is the other order, `lookup` first, which is blocked because `refill` needs a `spent` token that does not exist yet.

10. Vercel AI adapter: 64 tests

Sources in `packages/vercel-ai/src/`:

- `index.ts:37-75` `createPetriflowGate(nets, opts)`: builds `ctx` with `hasUI: !!opts.confirm` (46-49); `wrapTools()` creates a **new** `GateManager` on every call (53) and, if `messages` are given, replays them (55-60); returns `{ tools, systemPrompt, formatStatus, addNet, removeNet, manager }`.
- `wrap-tools.ts:28-91`: each tool's `execute` is replaced by: `handleToolCall` (51-54); on block throw `ToolCallBlockedError` (56-59); run the original; classify the result with `isToolResultError` and report it (62-76), treating a throwing classifier as an error (64-69, fail closed); on a thrown tool error report `isError: true` and re-throw (77-85). Tools without `execute` pass through untouched (37-40).
- `replay.ts:109-157` `extractReplayEntries(messages)`: index tool-call inputs by `toolCallId` (114-128), then walk tool messages; built-in error types `error-text`, `error-json`, `execution-denied` (44) always count as failures (136), otherwise the `isToolResultError` callback decides after unwrapping `{ type: "json", value }` (73-85).
- `errors.ts`: `ToolCallBlockedError` message `Tool 'x' blocked: <reason>` (line 8). This is the text the model sees, and the text lab 3's client prints as `BLOCKED:`.

`wrap-tools.test.ts` (46 tests), by describe block: **Free tools** (3), **Gated tools, allowed** (4, including the `toolCallId` reaching `onDecision`), **Gated tools, blocked** (4: throws, error fields, original `execute` never called), **Deferred transitions** (5: allowed immediately, success advances, thrown failure does not, the original error is re-thrown), **Manual transitions** (4: no `confirm` means blocked; `confirm` true or false; title and message content), **Schema-only tools** (2), **Multi-net composition** (4), **Registry mode** (4), **System prompt and status** (4), **Error propagation** (3: custom error classes survive), **Shadow mode** (3), **Edge cases** (4: a tool named like a place, ten rapid calls keep order, two tools on one transition), **Session**

isolation (2: two `wrapTools` calls do not share state, which is the "stateless gate" of the 0.2.0 design).

`replay.test.ts` (18 tests): **extractReplayEntries** (9) covers the message shapes above, including "isToolResultError does not override built-in error detection"; **wrapTools with messages** (3) shows the marking after replay (`tested: 1`); **isToolResultError (gate-level)** (6) shows the same classifier applied live and in replay, including the fail-closed case when the callback throws.

The adapter never looks at who produced the messages. The test "initializes gate state from message history" is, read the other way, the forgery demo in chapter 7.

11. The other adapters

Each adapter wraps the same `GateManager` for a different host. Their tests are short and mostly check wiring; the gate semantics are tested once in section 8.

11.1 `packages/agent-sdk` (21 tests, `src/gate.test.ts`)

`createPetriflowGate` (`src/index.ts:58`) returns `{ hooks, manager, systemPrompt, formatStatus }` where `hooks` has one `PreToolUse`, one `PostToolUse` and one `PostToolUseFailure` callback (135-138). PreToolUse ignores inputs whose `tool_name` is not a string (76-77), otherwise asks the manager (81) and, on block, returns `permissionDecision: "deny"` with the reason prefixed by the blocking net's name (86-95, using `findBlockingNet`). Tests cover free, gated, blocked, missing `tool_name`, deferred resolution through PostToolUse and PostToolUseFailure, in-process state across calls, abstain, multi-net, shadow mode, `onDecision`, and the three `confirm` cases. The `safeCodingNet` in this package (`src/nets/safe-coding.ts:11`) allows `Bash`; the one in `claude-code` locks it. No filesystem, no timers.

11.2 `packages/claude-code` (18 tests, `src/__tests__/hook.test.ts`)

The real hook executable `src/hook.ts` (stdin JSON to stdout JSON) is not imported; the test drives `GateManager` plus `src/state.ts` directly, "simulating hook logic". **"state persistence"** (7): `saveState` writes marking, meta and the pending list of every net to `/tmp/petriflow-claude-code-<session>.json` (`state.ts:21`, `44`); `restoreState` rebuilds the pending entries by transition name (69-79); `clearState` unlinks. **"PreToolUse gating"** (4): note the test named "unknown tool, deny (not free, not gated)" asserts `toBeUndefined()`, that is, allow; the comment in the test (lines 179-182) explains the net abstains. The title is wrong, the assertion matches the code. **"PostToolUse resolution"** (3), **"SessionStart"** (1), **"configure()"** (2: the hook command is `bun run <project>/node_modules/@petriflow/claude-code/src/hook.ts`), **"end-to-end with state persistence"** (1: three fresh managers imitate three hook processes).

Windows: the state path is a literal `/tmp/...`. Git Bash maps it to `C:\tmp`. On the fresh clone this file passed only because `C:\tmp` already existed on the machine; with the folder renamed away, the run was `10 pass, 8 fail` with `ENOENT ... /tmp/petriflow-claude-code-...json`. Create `C:\tmp` once (`mkdir -p /c/tmp`) before running it on a new laptop.

11.3 `packages/openclaw` (20 tests, `src/__tests__/index.test.ts`)

Package name `@petriflow/petriflow-gate`. `createPetriGatePlugin` (`src/index.ts:9`) registers `before_tool_call`, `after_tool_call`, `before_agent_start` and the commands `net-status`, plus `add-net`/`remove-net` only in registry mode (80). OpenClaw gives no tool call id, so the adapter mints `gate-<n>` ids (26-31) and correlates results through a per-tool FIFO queue; blocked calls are not enqueued (test "blocked tools don't leak IDs"). `before_tool_call` always passes `hasUI: false`, so every `manual` transition is denied in this host (test "blocks manual transitions (hasUI: false, auto-deny)"). The test API is a

hand-rolled mock; nothing from OpenClaw itself runs.

11.4 `packages/pi-extension` (19 tests, `src/__tests__/compose.test.ts`)

`composeGates` (`src/index.ts:11`) returns a function that registers `tool_call`, `tool_result`, `before_agent_start` and the same three commands on the pi-mono extension API. The `tool_call` handler forwards `ctx.hasUI` and `ctx.ui.confirm` (17-20) and returns the raw `GateDecision`. Tests mirror the manager tests (wiring, block, abstain, prompt text, dynamic add/remove, notifications) and end with the `communicateNet` plus `cleanupNet` composition. The handler logs `[petri-compose] tool_call=...` to stderr on every call; that is the noise you see in the run, not a failure. The three nets this package ships (`tool-approval`, `implement`, `nuke`) are the ones tested in section 8.1.

11.5 `packages/pi-assistant` (59 tests, 4 files)

Four hand-built nets with real `validateToolCall` and `onDeferredResult` logic, each tested through the single-net `handleToolCall`/`handleToolResult`:

- `cleanup.test.ts` (23): same backup-covers-destroy design as `nuke`, in `src/nets/cleanup.ts`. Two of its extraction tests fail on Windows (section 13).
- `communicate.test.ts` (14): `slack readMessages` is a deferred `observe` that records the channel; `sendMessage` is `send`, allowed only for an observed channel, one send per read (`validateToolCall`, `communicate.ts:139-157`).
- `deploy.test.ts` (12): `run-tests` and `build` are deferred self-loops that set `meta.testsPassed` and `meta.buildSucceeded`; `deploy-staging` is refused by `validateToolCall` unless tests passed; `deploy-prod` is `manual`; a staging deploy resets the flags.
- `research.test.ts` (10): a successful `web-fetch` (deferred) produces a `canShare` token; each `share` consumes one; three fetches allow three shares. The test "localhost curl is NOT treated as research" passes for a different reason than its name says: the regex at `research.ts:19` does match `http://localhost:3000/health`, the call is allowed as a deferred fetch, and `canShare` stays 0 only because no result is delivered.

12. Examples (46 tests) and comparisons (49 tests)

12.1 `examples/0X/agent.test.ts`

Each example folder has a `.rules` file, a runnable `agent.ts` (which calls an Anthropic model and is not imported by the test) and a test that loads the rules with `loadRules` and drives `createPetriflowGate(...).wrapTools()` with inline mock tools. No model, no network.

Folder	Rules	Tests	What they show
`01-file-management`	`require backup before delete, block rm`	7	free tools pass by abstention; `rm` is unreachable; delete needs a successful backup; failure does not unlock; the cycle repeats
`02-deployment`	lint before test, test before deploy, human approval before deploy, limit deploy 2	10	three nets judge one `deploy` call; the limit blocks in phase 1, before any approval prompt

Folder	Rules	Tests	What they show
`03-discord-bot`	`require discord.readMessages before discord.sendMessage`, limit 5	6	dot notation: one `discord` tool split by `input.action`; `addReaction` passes by abstention
`04-devops-assistant`	ten rules across Slack, email, deployment, files	23	cross-domain independence (a block in the file net does not touch Slack) and interleaving

`examples/05-openclaw-ops/ops-assistant.rules` (56 rules: 26 `require`, 16 `limit`, 14 `block`, plus 83 `map` lines) has no test. Chapter 5, loophole 3, is about what composing that many nets can do.

All four test files load their rules with `new URL("./x.rules", import.meta.url).pathname` (line 6 of each), which is the Windows crash in section 13.

12.2 `comparisons/n8n/comparison.test.ts` (11 tests)

Static analysis only. `analyseCalls` (`workflows/agent-benchmark/call-analysis.ts:49`) classifies a 12-step path through the agent-benchmark net: 4 steps are model choices, 1 is human, 7 are deterministic; the three parallel decisions can be batched into one call, so 2 model calls instead of 4, versus 11 for a ReAct loop (81.8% fewer, computed at lines 163-168 of call-analysis). The remaining tests reuse the four benchmark properties and assert that `comparisons/n8n/workflow.json` (20 nodes) has no equivalent structure. These are the numbers behind the website's comparison page; they are properties of the net definition, not measurements of a running agent.

12.3 `comparisons/openclaw-safety/src/scenarios.test.ts` (38 tests)

Four hand-built workflow nets (tool approval, message gating, budget and escalation, sandbox isolation, plus a locked variant). Every test is reachability or structure: a place that is never produced stays 0 in all reachable markings (the "dead transition" proofs), weighted sums are constant, terminal states are the declared ones, budgets stay within range. Guards in these definitions are dropped by `toNet`, so the analysis explores both sides of every guard.

13. The Windows failures, line by line, and the reversible patch

Thirteen problems, two causes, both in path handling. Linux and macOS are not affected by either (not run here; the cause is platform-specific by construction).

Cause 1: `node:path.normalize` **is the Windows flavour.** `packages/pi-extension/src/nets/nuke.ts:4` and `packages/pi-assistant/src/nets/cleanup.ts:4` import `normalize` from `node:path`. On Windows `normalize("src/")` returns `src\`, and the following `.replace(/\/+$/, "")` strips only forward slashes. So `extractBackupTarget("cp -r src/ /tmp/bak")` returns `src\` instead of `src`. Five tests fail:

```
(fail) path extraction > extracts backup targets           gate.test.ts:392   Expected: "src"
    Received: "src\"
(fail) path extraction > extracts destructive targets      gate.test.ts:400   Expected:
    "build" Received: "build\"
(fail) nukeNet > path match: backup src/ then rm src/generated/ -> allowed   gate.test.ts:550
(fail) extractBackupTarget > cp -r src/ /tmp/bak -> 'src'   cleanup.test.ts
(fail) extractDestructiveTarget > rm -rf build/ -> 'build'  cleanup.test.ts
```

Cause 2: a file URL's `pathname` is not a Windows path. `examples/0X/agent.test.ts:6` and `examples/0X/agent.ts:10` (22 in example 04) build the rules path as `new URL("./safety.rules", import.meta.url).pathname`, which on Windows is `/C:/Users/.../safety.rules`. `readFile` cannot open it:

```
# Unhandled error between tests
ENOENT: no such file or directory, open '/C:/.../examples/01-file-management/safety.rules'
```

The error happens at module load, so each of the four files is reported as `1 fail, 1 error` and none of its 46 tests runs.

Cause 3, only if `C:\tmp` is missing: `packages/claude-code/src/state.ts:21` writes to `/tmp/...`. With no `C:\tmp`, 8 of the 18 hook tests fail with ENOENT. `mkdir -p /c/tmp` fixes it without touching the code.

13.1 The patch

`labs/windows-test-fix.diff` in this repository changes six files: `import.meta.dir` instead of the URL pathname in the four example test files, and `posix.normalize` in the two nets (two lines each). The four example `agent.ts` files have the same path bug; they are covered by `labs/azure-examples.diff` (chapter 14), which can be applied alongside. Apply, test, and revert with:

```
git apply <path-to-this-repo>/labs/windows-test-fix.diff
bun test                       # observed: 639 pass, 0 fail, Ran 639 tests across 39 files
git checkout -- examples packages   # back to the untouched clone
```

The patch is for running the suite green on Windows while reading; it is not a proposal to change the project. The clone stays as it was after the `git checkout`.

The diff, for reading without applying:

```
--- a/examples/01-file-management/agent.test.ts
+++ b/examples/01-file-management/agent.test.ts
-  new URL("./safety.rules", import.meta.url).pathname,
+  `${import.meta.dir}/safety.rules`,
   (same one-line change in examples 02, 03, 04; agent.ts is in azure-examples.diff)

--- a/packages/pi-extension/src/nets/nuke.ts        (and
    packages/pi-assistant/src/nets/cleanup.ts)
+++ b/packages/pi-extension/src/nets/nuke.ts
-import { normalize } from "node:path";
+import { posix } from "node:path";
+const { normalize } = posix;
```

14. What the suite does not test

Facts checked by reading and by grep on the fresh clone:

- **The CLI binary.** No test executes `packages/cli/src/cli.ts`; its `--strict` logic (71-89) and the absence of any command other than `analyse` are untested.
- **The composition of rules.** `verifyNets` analyses each rule's net alone; no test (and no code) analyses the product of the composed nets. Two rules that together block everything are each "verified" (chapter 5, loophole 3).
- **Properties of compiled rules beyond a state count.** The verification tests assert `reachableStates` equals 2 or 5; no invariant, no terminal-state property, no deadlock check is computed for a compiled rule.

- **The `guard` field on a gated transition.** `GatedTransition` has `guard?: string | null` (`types.ts:16`); `grep -rn "\.guard" packages/gate/src` finds no read of it. A guard written on a skill net is never evaluated.
- **Replay provenance.** Replay tests prove what replay does with a given history; nothing tests or constrains where the history came from.
- **Per-action limit, prerequisite first.** The one per-action test calls the limited tool first; the order the website shows (`lookup`, then `send`) is untested and does not work (chapter 5, loophole 5).
- **Manual transitions in the engine.** `executor.step` fires `manual` transitions like any other; no test asserts that an engine `manual` transition waits for anyone.
- `examples/05-openclaw-ops`**.** The largest rules file in the repo has no test.
- **Real model behaviour.** Nothing here involves a model. Chapter 8's labs are where a model enters, and those are demos, not tests.

None of this is hidden; it is the shape of a 0.2.0 project. It is also the list of things a review should not claim the test suite proves.

15. Reading order and how to present a result

Suggested order, one sitting each:

1. `node_modules/petri-ts/dist/index.js` (208 lines), then `engine/src/types.test.ts`.
2. `engine/src/engine.ts`, `executor.ts`, `scheduler.ts`, with `scheduler.test.ts` open.
3. `workflows/agent-benchmark/definition.ts` and its test; run `bun test workflows/agent-benchmark` and match the printed summary to the nine assertions.
4. `gate/src/gate.ts`, `compose.ts`, `manager.ts`, with `compose.test.ts` open.
5. `rules/src/compiler.ts`, with the "verification" describe block open; then `vercel-ai/src/wrap-tools.ts` and `replay.ts`.
6. One adapter of your choice, and one example test.

When you paste a test result into a commit or a message, three lines carry the meaning:

```
Ran 639 tests across 39 files.      <- how much ran (597 on unpatched Windows)
 639 pass / 0 fail                  <- outcome
=== Agent Benchmark Analysis === Reachable states: 196 ...   <- the one printed proof
```

and the sentence that goes with them: the suite proves the engine fires what the net allows, the gate blocks what the nets forbid, and nine declared properties of one example net hold over its 196 reachable states; it does not prove anything about composed rules, the CLI's strict mode, or where a replayed history came from.